

MODULE 6

INTERNAL MEMORY BITS (M), GLOBAL VARIABLES AND RETENTIVE MEMORY

PLC Siemens TIA Portal Course

Module 6: MEMORY BITS AND MEMORY

Author: AUTOMATION AND ROBOTICS ENGINEER

Level: Basic-Intermediate

Software: Siemens TIA Portal

Language: Ladder Diagram (LAD)

Introduction

When developing industrial automation applications, it is essential to store, share, and preserve information within the PLC. To do this, Siemens provides different memory-management tools, among which internal memory bits (M), global variables, and retentive memory stand out.

These tools make it possible to implement more advanced logic, store operating states, share data between program blocks, and preserve important information even after a power outage.

In this module you will learn to correctly use these resources in TIA Portal V20, understanding how they work and their applications in real industrial environments.

Internal Memory Bits (M)

Internal memory bits, also known as Memory Bits or Flags, are internal PLC memory addresses that can be used as inputs, outputs, or auxiliary variables within the program.

They do not correspond to the controller's physical inputs or outputs; they exist only in the PLC's memory.

Examples:

M0.0

M0.1

M1.5

M10.0

Characteristics of internal memory bits

- They are internal PLC variables.
- They can store Boolean values.
- They help simplify programming.
- They are used to create latches and interlocks.
- They facilitate communication between program networks.

Industrial applications

- Starting and stopping motors.
- Safety interlocks.
- Machine states.
- Automatic sequences.
- Operating permissions.
- Operating modes.

Practical Example

A motor should stay on after a start push button is pressed.

This logic can be implemented using an internal memory bit.

When the push button is pressed:

$I0.0 \rightarrow M0.0$

The memory bit stores the operating state and subsequently drives:

$M0.0 \rightarrow Q0.0$

Example:

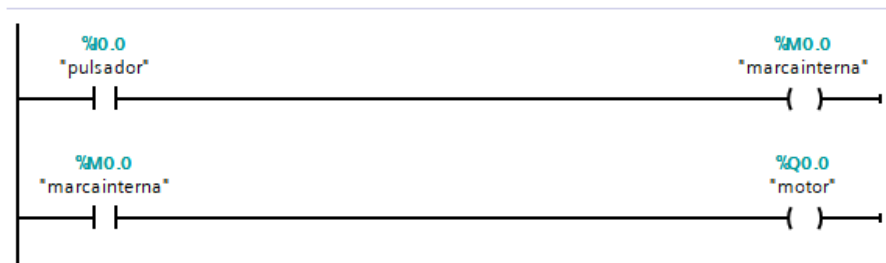


Diagram tags (original graphic, in Spanish): "pulsador" = push button · "marcainterna" = internal memory bit · "motor" = motor.

Advantages of using internal memory bits

- Reduces program complexity.
- Better logic organization.
- Easier maintenance.
- Ability to reuse information across different networks.

Global Variables

Global variables are variables that can be used from any block in the project, allowing information to be shared between different programs, functions, and data blocks.

In TIA Portal these variables are normally created in:

- PLC Tags.
- Global Data Blocks.

Examples of global variables

Motor_Running
Automatic_Mode
Parts_Produced
Process_Temperature
Belt_Speed

Industrial applications

- Sharing states between blocks.
- Storing machine parameters.
- Exchanging information between functions.
- HMI monitoring variables.
- Production variables.

Advantages of global variables

- Facilitate modular programming.
- Allow data reuse.
- Improve program scalability.
- Simplify communication between blocks.

Practical Example

A machine's automatic mode needs to be used by different blocks of the program.

The following variable is created:

`Automatic_Mode`

It can later be used in:

- Motor FC.
- Alarm FC.
- Safety FC.
- Production FC.

All blocks can access the same variable.

Retentive Memory

Retentive memory areas keep their value even after:

- Powering off the PLC.
- Restarting the CPU.
- Power outages.

This feature is essential in applications where important information must not be lost.

For example, a production count would be a retentive memory bit, since it has the Retain option checked — meaning that if the PLC is powered off, its count remains stored in the PLC's memory.

marcasejemplos ▸ PLC_1 [CPU 1214C DC/DC/DC] ▸ PLC tags ▸ Default tag table [32]

Default tag table

	Name	Data type	Address	Retain	Acces...	Writa...	Visibl...
1	 pulsador	Bool	%I0.0	☐	☒	☒	☒
2	 conteo_de_produccion	Bool	%M0.0	☒	☒	☒	☒
3	 motor	Bool	 %Q0.0	☐	☒	☒	☒
4	<Add new>			☐	☒	☒	☒

Screen tags (original graphic, in Spanish): "pulsador" = push button · "conteo_de_produccion" = production_count · "motor" = motor · Column headers Name / Data type / Address / Retain / Access / Write / Visible.

Industrial Applications

- Production counters.
- Hour meters.
- Number of parts manufactured.
- Production recipes.
- Machine parameters.
- Maintenance data.

Practical Example

A production counter records:

Total_Production = 15,800 parts

If a power outage occurs, once power is restored the value remains stored:

Total_Production = 15,800 parts

The data is not lost.

Configuring Retentive Memory in TIA Portal V20

Retentive variables can be configured in two ways:

Method 1: PLC Tags variables

By enabling the option:

Retain

Method 2: CPU Configuration

Path:

Device Configuration

↓

CPU Properties

↓

Retentive Memory

In this section you can configure:

- Retentive memory-bit area.
- Retentive data blocks.
- Amount of retained memory.

Real Applications of Retentive Memory

Industrial production

Save the total number of parts manufactured.

Pumping systems

Keep track of the number of operating hours.

Packaging machines

Save the number of boxes produced.

Preventive maintenance

Log operating cycles.

Recipe systems

Preserve production parameters.

Programming Best Practices

- Use descriptive names for memory bits and variables.
- Avoid excessive use of internal memory bits.
- Document every variable used.
- Use global variables only when necessary.
- Configure only important variables as retentive.
- Test operation after a CPU restart.
- Organize variables by category.

Practical Exercise

Design a program that meets the following conditions:

Exercise 1

Create an internal memory bit that latches the start of a motor using a seal-in circuit.

Exercise 2

Create a global variable named:

`Automatic_Mode`

and use it in two different blocks of the program.

Exercise 3

Create a production counter and store its value in retentive memory so it is not lost after a power outage.

Conclusion

Internal memory bits (M), global variables, and retentive memory are fundamental tools for developing professional Siemens PLC programs.

Using them correctly makes it possible to build more robust, organized, and reliable applications, facilitating communication between blocks and ensuring the preservation of critical process information.

Mastering these concepts is an essential step for any Siemens PLC programmer who wants to develop advanced industrial solutions using TIA Portal V20.

END OF MODULE 6

AUTOMATION STUDIO X

PLC Siemens TIA Portal Training System